Practical - 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and optional secure remember-me token.

A. Theory

1. Are sessions started and destroyed securely?

Answer: Yes, PHP sessions are started using session_start(). The session ID is regenerated after successful login using session_regenerate_id(true). During logout, session data is cleared, the session cookie is removed, and the session is destroyed using session_destroy().

2. Is password_verify() used correctly?

Answer: Yes, password_verify() compares the password entered by the user with the hashed password stored in the database. This allows secure password verification without storing plain-text passwords.

3. Is unauthorized dashboard access blocked?

Answer: Yes, dashboard pages check whether the user has an active authenticated session. If a user is not logged in, they are redirected to the login page. Role-specific pages also verify the user's role before granting access.

4.Is session timeout implemented?

Answer: Yes, a session timeout can be implemented by tracking the user's last activity timestamp. If the user remains inactive beyond the configured time limit, the session is cleared and the user must log in again.

5. Are admin and student users redirected to appropriate dashboards?

Answer: Yes, role-based redirection sends administrators to the admin dashboard and students to the student dashboard after successful authentication.

B. Implementation

Verify password and regenerate session ID

```
session_start();

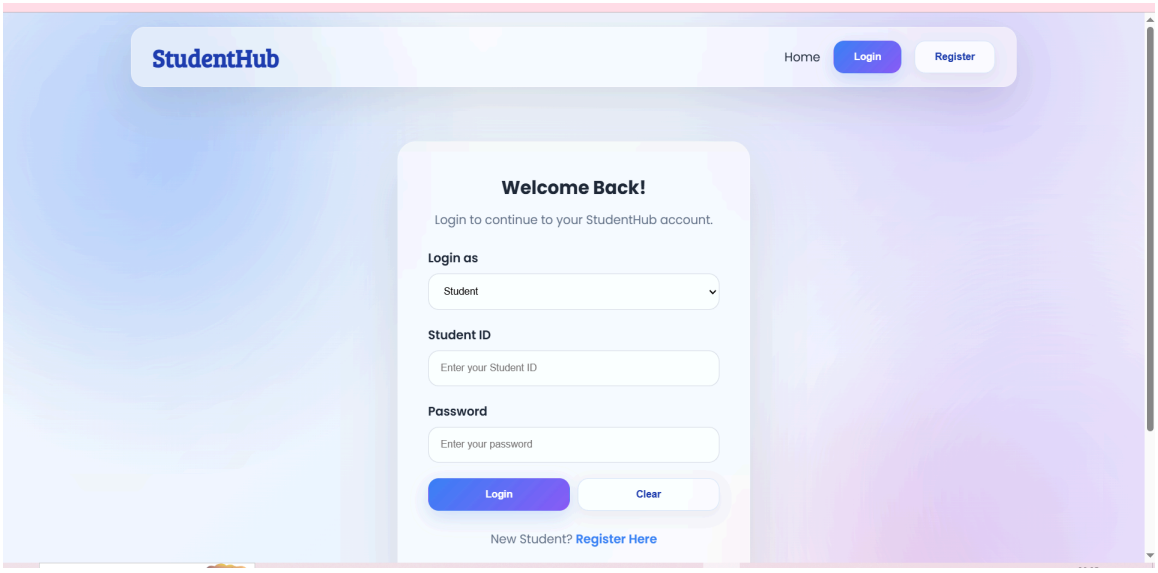
if (password_verify($password, $user['password'])) {
    session_regenerate_id(true);

    $_SESSION['user_id'] = $user['id'];
    $_SESSION['role'] = $user['role'];

    if ($user['role'] === 'admin') {
        header("Location: admin_dashboard.php");
    } else {
        header("Location: student_dashboard.php");
    }
    exit;
} else {
    echo "Invalid email or password.";
}
```

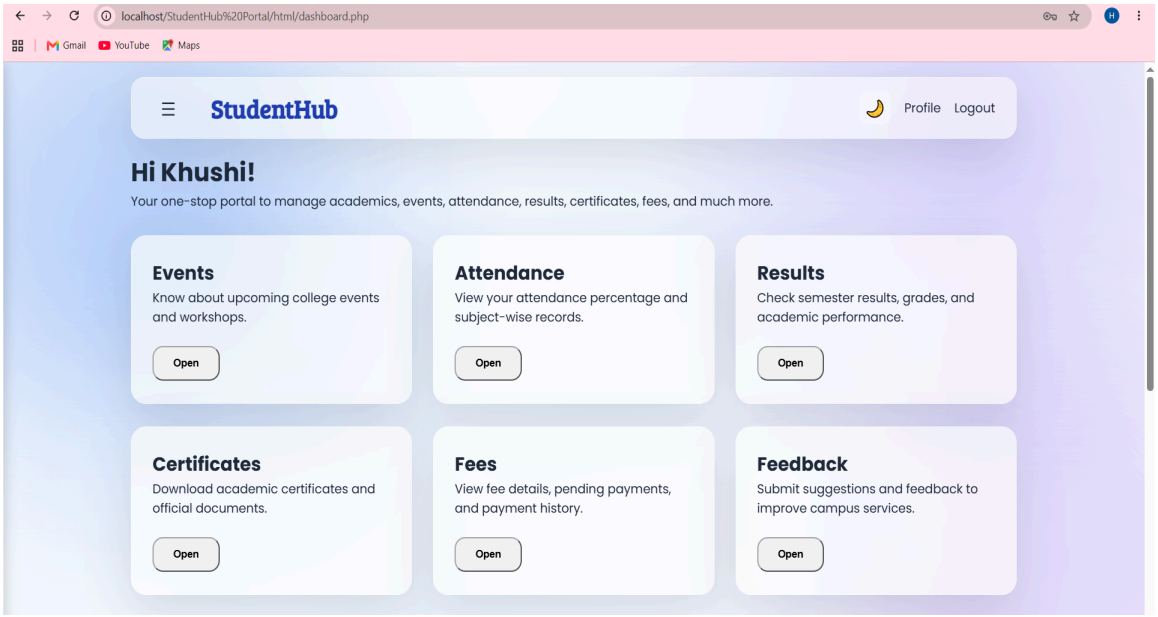
C. Evidence of Work (Screenshots)

Figure 1.1 Login page

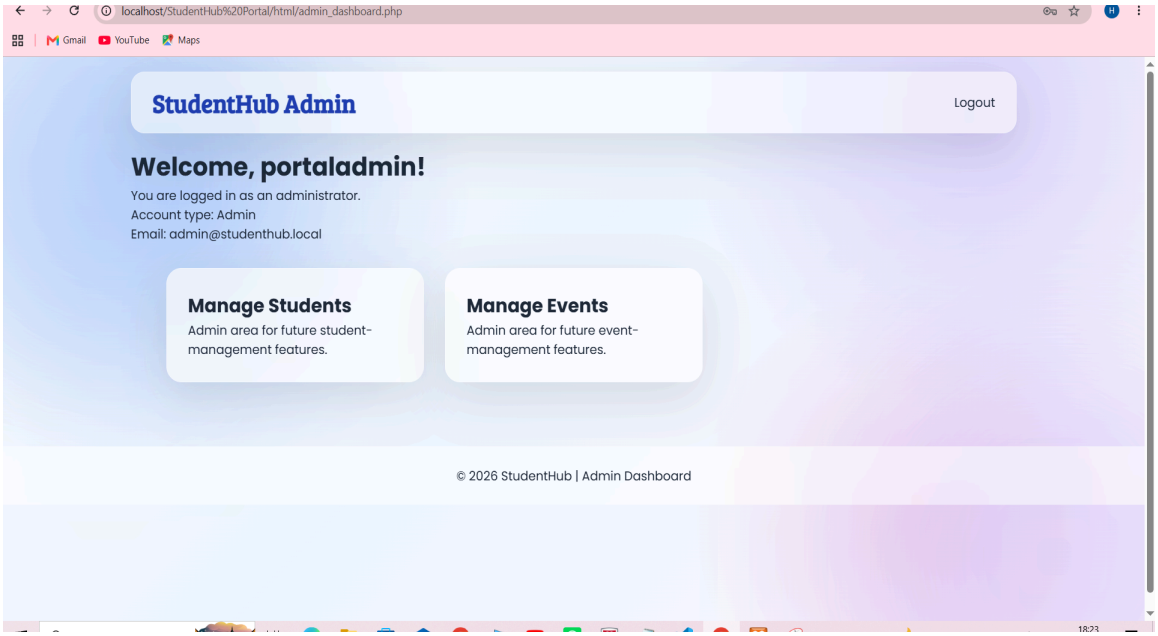


Description: Login asks for role and if successful, navigates to related dashboard

Figure 1.2 Dashboard

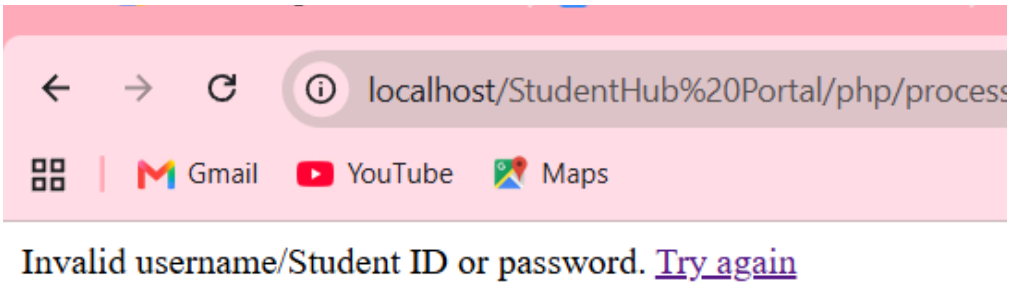


Description: Student's dashboard

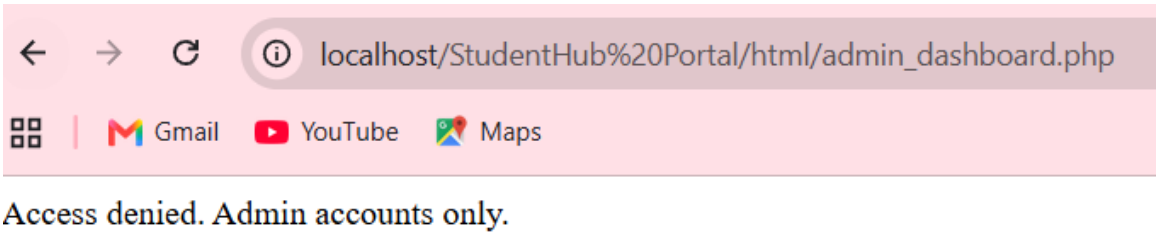


Description: Admin's Dashboard

Figure 1.3 Message



Description: Shows message when typed wrong id or password



Description: Shows message when trying to access admin dashboard while logged in as student

D. GitHub Repository Link:

https://github.com/heerce27/StudentHub-Portal.git

E. Challenges Faced:

- Configuring role-based redirection for administrators and students.

F. Learning Outcomes:

- Learned how to implement login and logout functionality using PHP sessions.
- Understood password verification and secure authentication practices.

Faculty Remarks: